

Markov Chains: Properties Needed for Transition-Informed Associative Sequential Memory

Motivation

The goal is a Dense Associative Memory (DAM) where the gradient descent update is informed by the *previous* valley/state, i.e. the network encodes not just individual attractors $\zeta_1, \zeta_2, \dots$ but the *transition structure* between them (e.g. $\zeta_i \rightarrow \zeta_{i+1}$, or more generally $\zeta_i \rightarrow \zeta_{i+k}$). This is formally a Markov chain over the space of attractor states. This document collects the Markov chain machinery relevant to encoding, training, and analyzing that transition structure — including the “ $i + 2$ ” (higher-order / skip-transition) extension you raised.

1 Basic Definitions

Definition 1 (Markov Chain). *A sequence of random variables $X_0, X_1, X_2, \dots$ taking values in a state space $\mathcal{S}$ is a (first-order, discrete-time) Markov chain if it satisfies the **Markov property**:*

$$P(X_{t+1} = s_{t+1} \mid X_t = s_t, X_{t-1} = s_{t-1}, \dots, X_0 = s_0) = P(X_{t+1} = s_{t+1} \mid X_t = s_t).$$

That is, the future depends on the past only through the present state. In your setting, $\mathcal{S}$ is the set of stored attractor/memory patterns $\{\xi^1, \dots, \xi^M\}$, and X_t is which attractor the network occupies at (discrete) time step t .

Definition 2 (Transition Matrix). *For a finite state space $|\mathcal{S}| = M$, the transition matrix $T \in \mathbb{R}^{M \times M}$ has entries*

$$T_{ij} = P(X_{t+1} = j \mid X_t = i).$$

*T is **row-stochastic**: every row sums to 1, $\sum_j T_{ij} = 1$, and $T_{ij} \geq 0$.*

This T is exactly the object your asymmetric coupling / transition-edge weights should be approximating. In a deterministic sequence (your baseline case), T is a permutation matrix: $T_{i,i+1} = 1$ and all other entries in row i are zero. The moment you allow probabilistic branching between sequences (e.g. two learned sequences that share a sub-pattern), T becomes genuinely stochastic, and this is where Markov chain theory earns its keep.

A crucial distinction: the transition matrix T is *not* the same object as the network’s coupling matrix J

This distinction is easy to blur informally, so it is worth stating precisely. If your network’s weights (e.g. an asymmetric Hebbian coupling $J_{ij} = \sum_\mu \xi_i^{\mu+1} \xi_j^\mu$) are used to drive sequence transitions, J is **not itself** a Markov transition matrix, even though it encodes transition structure. Concretely:

- T_{ij} must satisfy $T_{ij} \in [0, 1]$ and $\sum_j T_{ij} = 1$ for every row i — this is definitional, not an approximation. Any genuine Markov transition matrix is row-stochastic by construction.

- J_{ij} , built from sums of products of bipolar pattern entries (± 1), can be any real number — including negative (inhibitory couplings, which are essential for driving oscillators *away* from incorrect phases) — and its rows have no fixed sum.

So J is a **synaptic weight matrix that encodes** which transitions are learned, while T is the **normalized probability object** describing the resulting dynamics. If you want an actual stochastic matrix out of a trained network (e.g. for the diagnostics in Section 7), you must derive it — either by softmax-normalizing rows of J after shifting to non-negative values, or (preferably) by empirically simulating the free-running network many times, recording which attractor state follows which, and normalizing the resulting transition counts into a proper row-stochastic $\hat{T}$. Treating J itself as already satisfying the stochasticity constraint is a category error worth avoiding throughout this analysis.

n -step transitions

The probability of being in state j after n steps starting from i is given by matrix powers:

$$P(X_{t+n} = j \mid X_t = i) = (T^n)_{ij}.$$

This is precisely the mathematical object underlying your “ $i + 2$ ” idea: if T encodes single-step transitions $i \rightarrow i + 1$, then T^2 encodes two-step transitions $i \rightarrow i + 2$. Concretely,

$$(T^2)_{ik} = \sum_j T_{ij} T_{jk}.$$

If you want the network’s gradient update to be informed of the valley two steps prior (not just one), you are, in effect, asking the learning rule to fit T^2 (or a weighted combination of T and T^2) rather than T alone. This is a form of higher-order Markov dependency — see Section 4.

2 The Chapman–Kolmogorov Equation

This is the master identity behind all n -step calculations, and it is the reason matrix powers work:

$$P(X_{t+n} = j \mid X_t = i) = \sum_{k \in \mathcal{S}} P(X_{t+m} = k \mid X_t = i) P(X_{t+n} = j \mid X_{t+m} = k), \quad 0 \leq m \leq n.$$

In matrix form: $T^{(n)} = T^{(m)} T^{(n-m)}$. This is what justifies decomposing a long sequence-recall path into shorter sub-paths — relevant if your Kuramoto network recalls a sequence by chaining several two-step ($i \rightarrow i + 2$) hops rather than single-step hops.

3 Stationary Distributions

Definition 3 (Stationary Distribution). *A row vector π (a probability distribution over $\mathcal{S}$) is **stationary** for T if*

$$\pi T = \pi, \quad \sum_i \pi_i = 1, \quad \pi_i \geq 0.$$

Equivalently, π is a left eigenvector of T with eigenvalue 1.

Why this matters for associative memory: if your Kuramoto/DAM network is left running freely (not clamped to a specific target sequence), the long-run fraction of time it spends in each attractor state is exactly π . If the network shows strong drift toward a small subset of attractors regardless of the initial probe, that is evidence the effective transition matrix induced by your gradient-descent-trained weights has a stationary distribution concentrated on those states — potentially undesired if you want all sequence states to be visited with intended frequency. This is a direct diagnostic tool: compute (or estimate empirically via simulation) π for your trained network and compare to the target sequence frequencies.

Why π exists: a spectral view

The cleanest way to see where π comes from is via the eigenstructure of T . Since T is a non-negative matrix with every row summing to 1, the **Perron–Frobenius theorem** guarantees T has a largest eigenvalue $\lambda_0 = 1$ (with right eigenvector the all-ones vector $\mathbf{1}$, since $T\mathbf{1} = \mathbf{1}$ trivially). The corresponding *left* eigenvector, normalized to sum to 1, is π :

$$\pi T = \mathbf{1} \cdot \pi.$$

Every other eigenvalue satisfies $|\lambda_k| \leq 1$. Decomposing an arbitrary starting distribution ρ_0 in the eigenbasis of T and repeatedly multiplying by T causes every component except the $\lambda_0 = 1$ component to shrink (strictly, if $|\lambda_k| < 1$) or merely rotate in the complex plane (if $|\lambda_k| = 1$ but $\lambda_k \neq 1$) — only the $\lambda_0 = 1$ component survives unchanged. This is precisely why repeated application of T drives *any* initial distribution toward π : π is the unique direction in distribution-space that T leaves completely invariant.

A worked example, and a correction to the periodic case

For a purely deterministic cyclic sequence $\xi^1 \rightarrow \xi^2 \rightarrow \dots \rightarrow \xi^M \rightarrow \xi^1$, T is a cyclic permutation matrix. Check directly that the *uniform* distribution $\pi_i = 1/M$ satisfies $\pi T = \pi$: since every column of a permutation matrix sums to exactly 1, $(\pi T)_j = \sum_i \frac{1}{M} T_{ij} = \frac{1}{M} \sum_i T_{ij} = \frac{1}{M}$. So π **does exist** for a periodic chain — existence only requires **irreducibility** (every state reachable from every other), not aperiodicity. What aperiodicity buys you is the *pointwise* limit $T_{ij}^n \rightarrow \pi_j$ (Property 1 above). For a periodic chain, this pointwise limit fails — T^n instead cycles forever through M distinct permutation matrices — but the weaker **Cesàro-averaged** version of the ergodic theorem still holds:

$$\frac{1}{N} \sum_{n=0}^{N-1} T_{ij}^n \longrightarrow \pi_j \quad \text{as } N \rightarrow \infty,$$

i.e. the *time-averaged* fraction of visits to state j converges to $\pi_j = 1/M$ even though the instantaneous state never settles down. This is the precise statement underlying the earlier claim about limit cycles: your ideal sequence-recall network should show *uniform time-averaged occupancy* across all M stored patterns (a well-defined, computable π) while never actually converging to a fixed instantaneous distribution — both things are true simultaneously, and neither contradicts the other.

Property 1 (Existence and Uniqueness). *If T is **irreducible** (every state reachable from every other state), a unique stationary distribution π exists (Perron–Frobenius). If additionally T is **aperiodic** (no forced cyclic return pattern), then*

$$\lim_{n \rightarrow \infty} T_{ij}^n = \pi_j \quad \text{for all } i.$$

*This stronger, pointwise statement is the **ergodic theorem** for Markov chains.*

Practically: a network trained to recall a single deterministic cyclic sequence is periodic with period M and will *not* exhibit the pointwise convergence in the Property above, but it will (and should) exhibit uniform Cesàro-averaged occupancy. If instead you observe uneven long-run occupancy (some patterns visited far more than $1/M$ of the time, on average over many steps) that is the real signature of degraded sequence learning — not the absence of pointwise convergence, which is expected and desired for clean recall.

4 Order of a Markov Chain and Your “ $i + 2$ ” Idea

Definition 4 (k -th Order Markov Chain). *A chain is k -th order Markov if*

$$P(X_{t+1} = s_{t+1} \mid X_t, X_{t-1}, \dots, X_0) = P(X_{t+1} = s_{t+1} \mid X_t, X_{t-1}, \dots, X_{t-k+1}),$$

i.e. the next state depends on the last k states, not just the last one.

Your “ $i+2$ ” framing can mean one of two different things, worth disambiguating precisely since they lead to different implementations:

- (a) **Skip-transitions (still 1st order, but predicting further ahead):** the network’s gradient step at pattern ξ^i is informed by ξ^{i+2} ’s valley in addition to ξ^{i+1} ’s. This is naturally captured by including T^2 (or a mixture $\alpha T + (1 - \alpha)T^2$) as an auxiliary target in the loss — i.e. the coupling/weight update rule includes a term that rewards stability not just at the immediate successor but at the successor-of-successor.
- (b) **2nd-order Markov dependency:** the transition probability into ξ^{i+1} genuinely depends on *both* ξ^{i-1} and ξ^i , not on ξ^i alone. This is the classical trick for de-aliasing repeated subsequences (if pattern B appears in two different contexts, $A \rightarrow B \rightarrow C$ and $D \rightarrow B \rightarrow E$, a 1st-order chain cannot disambiguate what follows B ; a 2nd-order chain conditioning on the pair (A, B) vs. (D, B) can).

Standard reduction to 1st order: any k -th order chain over state space $\mathcal{S}$ can be rewritten as a 1st-order chain over the expanded state space $\mathcal{S}^k$ (i.e., states become k -tuples / overlapping windows of the original states). Concretely, define $Y_t = (X_t, X_{t-1}, \dots, X_{t-k+1})$; then Y_t is first-order Markov. **This is directly implementable in your architecture:** instead of storing single patterns ξ^i as attractors, store concatenated pattern-pairs (ξ^{i-1}, ξ^i) as the attractor state, and let the (now 1st-order) transition structure between these pair-states encode what was previously 2nd-order structure. The cost is a larger effective state space (M^k instead of M), which interacts with your DAM’s superlinear capacity scaling in n (the interaction vertex) — arguably a good trade if your capacity headroom is large.

5 Detailed Balance and Reversibility

Definition 5 (Detailed Balance). *T (with stationary distribution π) satisfies detailed balance if*

$$\pi_i T_{ij} = \pi_j T_{ji} \quad \text{for all } i, j.$$

*A chain satisfying this is called **reversible**.*

Classical symmetric-weight Hopfield networks are the discrete-time, energy-descent analogue of a *reversible* process — this is exactly why they always converge to a fixed point rather than cycling (symmetric weights $\Rightarrow$ a Lyapunov/energy function exists $\Rightarrow$ no sustained directed motion). To get *sequences* at all, you need to explicitly break detailed balance: the asymmetric $i \rightarrow i + 1$ coupling in classical sequence-Hopfield models (Sompolinsky–Kanter and successors) is precisely a controlled violation of detailed balance, injecting net “flow” around the attractor landscape rather than allowing it to settle. Practically: if you check whether your learned Kuramoto/DAM coupling matrix is (anti)symmetric, that tells you directly whether the network is structurally capable of sustained sequence traversal versus only fixed-point recall.

6 Absorbing States and Hitting Times

Definition 6 (Absorbing State). *State i is absorbing if $T_{ii} = 1$ (once entered, never left).*

Definition 7 (Hitting Time). *The hitting time $\tau_j = \min\{t \geq 0 : X_t = j\}$ is the first time the chain reaches state j . The expected hitting time from i , $h_i = \mathbb{E}[\tau_j \mid X_0 = i]$, satisfies the linear system*

$$h_i = 1 + \sum_{k \neq j} T_{ik} h_k, \quad h_j = 0.$$

Relevance: if your sequence-recall dynamics should terminate at a final pattern (rather than cycle indefinitely), that final pattern should be modeled as (near-)absorbing, and expected hitting time gives you a principled way to reason about *how many relaxation/oscillation steps* the network needs before reliably reaching the end of a stored sequence — a natural diagnostic to compare against the number of Kuramoto integration steps you actually use.

7 Linking the Machinery to the Weight Matrix J_{ij}

Everything above has been phrased abstractly in terms of T . This section closes the loop by connecting it explicitly to the neural coupling matrix $J_{ij} = \sum_{\mu=1}^P \xi_i^{\mu+1} \xi_j^\mu$ derived earlier for the sequential Kuramoto/DAM network, and by examining two structural special cases that change the picture: sequences that **cycle back on themselves**, and sequences that are **self-referencing** (a pattern recurs or maps to itself within the trained sequence).

J as an operator on pattern-overlap space

Write the $N \times P$ pattern matrix $\Xi = [\xi^1 \mid \xi^2 \mid \cdots \mid \xi^P]$, and let Ξ_{shift} be the same matrix with columns cyclically advanced by one position (column μ of Ξ_{shift} is $\xi^{\mu+1}$). Then

$$J = \Xi_{\text{shift}} \Xi^\top.$$

If the patterns are mutually orthogonal (a standard simplifying assumption, e.g. $\frac{1}{N} \xi^\mu{}^\top \xi^\nu = \delta_{\mu\nu}$), then acting on the P -dimensional subspace spanned by the patterns, J behaves like the $P \times P$ **cyclic permutation matrix** C with $C_{\mu, \mu+1} = 1$ and all other entries 0 (indices mod P). This is the one setting where J and a genuine Markov transition matrix T coincide exactly: C is itself row-stochastic (it is a permutation matrix, a special case of a doubly-stochastic matrix), so in the idealized orthogonal-pattern limit, $T = C$ precisely — no empirical estimation or normalization step is needed, in contrast to the general case flagged in Section 1.

Cyclic sequences and the spectrum of C

When the training sequence wraps around ($\xi^{P+1} := \xi^1$), C is exactly the cyclic shift on P symbols, and its eigen-decomposition is standard and explicit: the eigenvalues are the P -th roots of unity,

$$\lambda_k = e^{2\pi i k/P}, \quad k = 0, 1, \dots, P-1,$$

with eigenvectors the discrete Fourier modes $(v_k)_\mu = e^{2\pi i k \mu/P}$. This gives a direct, quantitative bridge between the Markov-chain description and the Kuramoto network’s continuous phase dynamics:

- The $k = 0$ mode ($\lambda_0 = 1$) is the uniform vector — exactly the stationary distribution π derived in Section 3’s worked example, now recovered as the trivial (DC) Fourier component of C .
- The $k = 1$ mode corresponds to a single full traversal of the P -cycle: the fundamental frequency of the recall dynamics. This is the discrete-time analogue of a rotating wave in the continuous Kuramoto system — the same structure that produces the sustained circulation (limit cycle) discussed in Section 5, now derived from the spectrum of J rather than only asserted from the detailed-balance argument.
- Higher k modes correspond to faster sub-harmonics (e.g. $k = 2$ effectively traverses the cycle twice as fast in Fourier phase) — these are the natural candidates to inspect if you want to understand or exploit the “skip-transition” (T^2 -type) structure from Section 1 directly from the network’s spectrum, rather than by empirical simulation.

Practically: if you diagonalize your trained J (restricted to the pattern-overlap subspace) and its eigenvalues cluster near the P -th roots of unity, that is strong quantitative evidence the network has learned a clean single-cycle sequence; eigenvalues that drift off the unit circle indicate either decaying transitions (crosstalk killing the cycle) or growing ones (instability), both worth flagging.

Self-referencing sequences

A “self-referencing” sequence is one where a stored pattern recurs — either (a) a single pattern explicitly maps to itself, $\xi^{\mu+1} = \xi^\mu$ for some μ (the sequence is trained to *pause* at that pattern), or (b) the same physical pattern is reused at two different points in the intended trajectory (aliasing), e.g. $A \rightarrow B \rightarrow C \rightarrow D \rightarrow B \rightarrow C \rightarrow D \rightarrow \dots$, so the trajectory looks like a short transient tail feeding into a repeating cycle (a “lasso” shape in state space) rather than a pure cycle.

Both cases break the clean circulant picture above, but in different, informative ways:

- **Case (a), an explicit self-loop:** this inserts $T_{\mu\mu} > 0$ directly. A single self-loop makes the chain **aperiodic** (the gcd defining the period, Section 3, now includes a return time of 1), so the strong pointwise ergodic theorem (Property 1) applies: $T^n \rightarrow \mathbf{1}\pi^\top$, genuine convergence rather than perpetual cycling. This is a clean, principled way to convert a purely periodic network into one with a well-defined asymptotic visiting frequency — useful if you deliberately want a pattern to be revisited more often (e.g. modeling the persistence-time mechanism from the BCPNN literature, where a pattern is designed to remain active for several steps before the sequence continues).
- **Case (b), aliasing / a repeated pattern:** here the state-classification machinery of Markov chains applies directly. States split into a **transient class** (the tail A , visited once and never

returned to) and a single **recurrent communicating class** (the cycle $B \rightarrow C \rightarrow D \rightarrow B$). The stationary distribution π , again guaranteed by Perron–Frobenius, assigns exactly zero probability mass to every transient state and is supported entirely on the recurrent cycle — asymptotically, the network forgets it ever visited A at all. This is precisely the sequence-disambiguation problem from Section 4 viewed spectrally: naive training cannot tell whether being at B means “the network just came from A ” or “the network is mid-cycle, having come from D ,” because J built from raw pattern overlaps has no way to encode that context. The expected hitting time machinery from Section 5 is the right tool to quantify how long the transient tail persists before the network locks into the recurrent cycle, and the k -tuple state-expansion fix from Section 4 remains the principled resolution: expand the state representation so that “arriving at B from A ” and “arriving at B from D ” are encoded as distinct states before computing J , restoring a clean, disambiguated circulant (or near-circulant) structure on the expanded space.

8 Practical Checklist for Your Model

1. **Extract the empirical stochastic transition matrix** of your trained network (not its weight matrix J — see the T vs. J distinction in Section 1): for each stored pattern ξ^i , probe the network and record which attractor it relaxes/flows to next; tabulate empirical frequencies and normalize each row to sum to 1 to build a genuine $\hat{T}$.
2. **Compare $\hat{T}$ against the target**: for a clean deterministic sequence you expect $\hat{T} \approx$ a permutation matrix; measure deviation (e.g. Frobenius norm, or KL divergence row-wise) as your central metric of “how well transitions were learned,” analogous to the paper’s use of average accuracy.
3. **Check symmetry of coupling**: symmetric coupling forbids sustained sequences (Section 5); confirm your learned couplings are meaningfully asymmetric before troubleshooting elsewhere.
4. **If disambiguating repeated sub-patterns**: consider the k -tuple state-expansion trick (Section 4) rather than trying to hand-craft an ad hoc “ $i + 2$ ” correction term.
5. **Use $T^2, T^3, \dots$ as auxiliary loss targets** if you want gradient descent informed by valleys further down the sequence, not just the immediate successor — a direct, principled way to implement “lookahead” association without changing the state representation.
6. **Track the stationary/periodic behavior**: simulate the free-running network for many steps and check whether it settles into the intended limit cycle (period M , visiting all stored patterns with intended relative frequency) or collapses onto a subset — this is your practical stand-in for computing π or checking periodicity analytically.